

Distributed Systems HW3 – Section 2

Weather Data Analytics (HW2 Q8) using MapReduce and gRPC

Team **XX**: Rajai Tarun Kanaiyalal (roll no.) Teammate name (roll no.)

1 Q1: Batch analytics using MapReduce

In HW2 we solved the weather analytics problem (Q8) with a sequential program and with MPI. Here we solve the same problem, with the same input and output, using MapReduce, and then compare it with our MPI program. The code and the full run instructions are in `q1_mapreduce/README.md`.

1.1 How we ran it

The assignment asks for Hadoop Streaming. Hadoop is not installed on the RCE cluster (there is no Hadoop module), and the course told us to use a Slurm script until it is fixed. So we wrote the mapper, combiner and reducer as normal C++ programs that read from stdin and write `key<TAB>value` lines to stdout. This is exactly what Hadoop Streaming expects, so the same programs will also run on Hadoop without any change. Our Slurm script does the same steps that Hadoop would do:

1. **Split**: cut the input file into P pieces (one per mapper).
2. **Map**: P tasks run in parallel, each doing `mapper | sort | combiner` on its own piece.
3. **Shuffle**: the master merges all the sorted combiner outputs into one sorted file.
4. **Reduce**: one reducer reads that file and prints the final answer.

1.2 Our MapReduce design

For every record, the mapper prints three lines, using these keys:

Key	Value	How two values are combined
G	count, sums, min/max, extreme count, hottest, coldest	add, min/max, hotter/colder
S<station>	count, temperature sum, rainfall sum	add
I<interval>	count (interval = timestamp / 60)	add
K	the K from the header line	keep

The main idea: **one record is treated as a small partial result with count 1**. So the mapper output, the combiner input and the combiner output all have the same format, and the combiner and reducer only need to know how to merge two lines with the same key. Since the merges are just additions, mins and maxes, the order does not matter. The answer is the same no matter how the input is split or how many times the combiner runs. We never merge averages, only sums and counts, and divide at the very end.

Top- K stations and the busiest interval can only be decided after *all* the counts are merged (the same bug we found in HW2 when each process kept its own top- K). That is why we use **one reducer**: it sees every station and every interval.

1.3 Important decisions

Table 1 lists the main decisions we took, why, and what effect we measured.

1.4 Correctness

- **Local tests (69, all pass)**: the 9 HW2 test cases with expected outputs (ties, extreme values, $K > S$, $N = 0$) give the exact expected output. All 13 HW2 inputs plus generated 100K and 1M inputs, each split into 1, 2, 4 and 8 pieces, give the same output as the HW2 sequential program.

Decision	Why	Effect
Use a combiner	The mapper prints 3 lines per record, and the same keys repeat a lot inside one piece.	10M records: 30M $\rightarrow$ 7M lines sent to the reducer; 52 s $\rightarrow$ 16 s.
Only one reducer	Top- K and busiest interval need all the data in one place.	Reducer takes only 0.5–3.3 s.
Mapper prints the numbers exactly as read	Printing them as doubles turned 24.49 into 24.489999999999998.	Mapper output 2.1 $\times$ smaller.
Read numbers with <code>strtod</code>	<code>stringstream</code> was very slow for millions of lines.	Combiner 2.9 $\times$ faster.
LC_ALL=C <code>sort</code>	The normal <code>sort</code> can ignore the tab and mix up keys like <code>\$1</code> and <code>\$10</code> .	Correct grouping.
Pipe <code>mapper sort combiner</code>	The full mapper output for 50M records is 7.5 GB, but our disk quota is 10 GB.	Nothing big is written to disk.
Same data, same node as MPI	A fair comparison with HW2.	Differences come from the model, not the setup.

Table 1: Main design decisions.

- **Cluster:** the same output as sequential on one node and on two nodes, and all 44 benchmark runs are correct.

One small rounding difference. From 10M records onwards, the last digits of `TOTAL_RAINFALL` differ, e.g. 249966638.539998 (sequential) vs 249966638.540026 (MapReduce). The true value is 249966638.54, so both are slightly off. A `double` only keeps about 15–16 digits, and adding 10 million numbers in a different order gives a slightly different rounding. Our HW2 MPI program shows the same difference against sequential when it uses 1, 2 or 4 workers. So our checker (`compare_outputs.py`) requires counts, ids and timestamps to match exactly, and allows decimal numbers to differ only by a relative 10^{-10} . Every run passed: 15 matched exactly and 25 differed only by this rounding.

1.5 Experimental setup

We used the HW2 dataset generator with seed 42, $K = 10$ and $S = 100$, and input sizes of 1M, 10M, 20M and 50M records (50M records is 2.1 GB). We ran MPI with 1, 2, 4 and 8 workers and MapReduce with 1, 2, 4 and 8 mappers. All runs of one benchmark were in the same Slurm job, on one node, on the same data. We measured the total time of each run (including reading the input), the time of each MapReduce stage, and the peak memory of every process.

1.6 Results

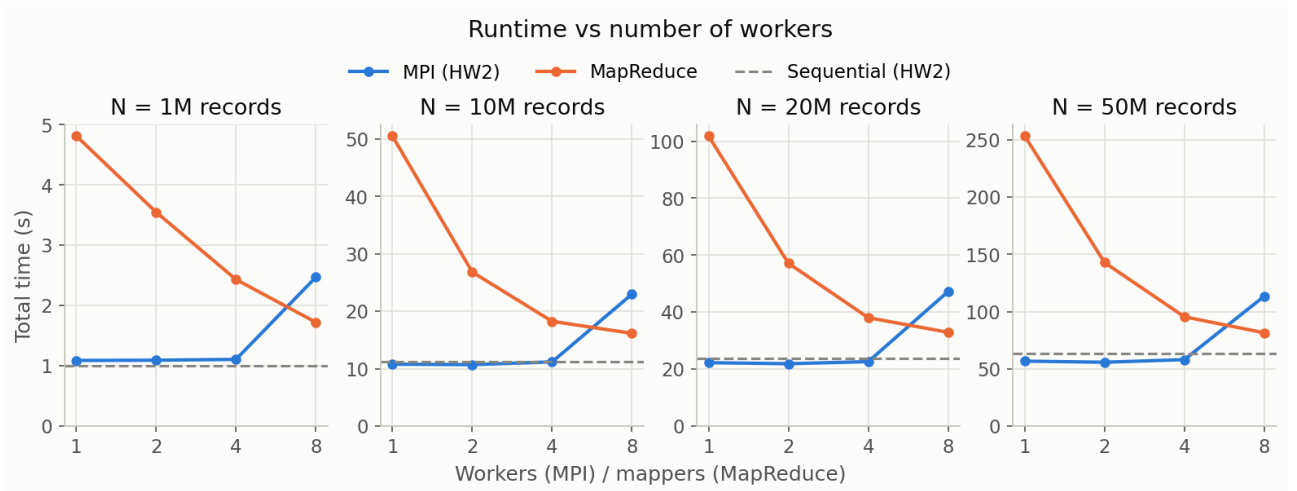


Figure 1: Total time vs number of workers/mappers. Dashed line: HW2 sequential.

N	Seq	MPI 1	MPI 2	MPI 4	MPI 8	MR 1	MR 2	MR 4	MR 8
1M	1.00	1.09	1.09	1.11	2.47	4.81	3.55	2.43	1.72
10M	11.22	10.78	10.70	11.19	22.98	50.51	26.83	18.24	16.16
20M	23.87	22.25	21.88	22.61	47.21	101.77	57.13	38.03	32.91
50M	63.65	56.74	55.78	58.06	113.48	253.44	142.90	95.46	81.50

Table 2: Total time in seconds (MR = MapReduce).

MPI is faster overall (Fig. 1, Table 2). At 50M records the best MPI run takes 55.8s and the best MapReduce run takes 81.5s. Both grow linearly with the input size. MapReduce does much more work per record. MPI reads each record once into a small binary struct and sends structs. MapReduce turns each record into 3 text lines (151 bytes, 3.6 times bigger than the input), sorts all these lines, and then reads the numbers back from text in the combiner. With only one mapper, MapReduce is 4 times slower than sequential (253s vs 64s).

MPI does not get faster with more workers. From our HW2 timing logs, at 50M records rank 0 spends about 46s of the 56s just *reading the input file alone*. The workers can only start after that, so adding more workers only speeds up the small compute part. With 8 workers the time even doubles, and the HW2 log shows that it is rank 0’s reading that becomes twice as slow. Our best guess is that the 8 waiting MPI processes keep the CPUs busy while they wait, which slows down rank 0.

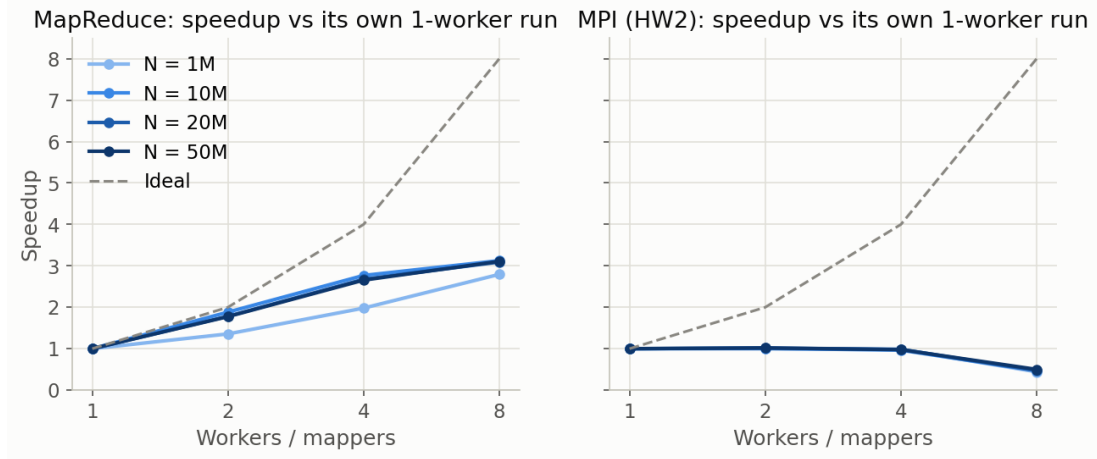


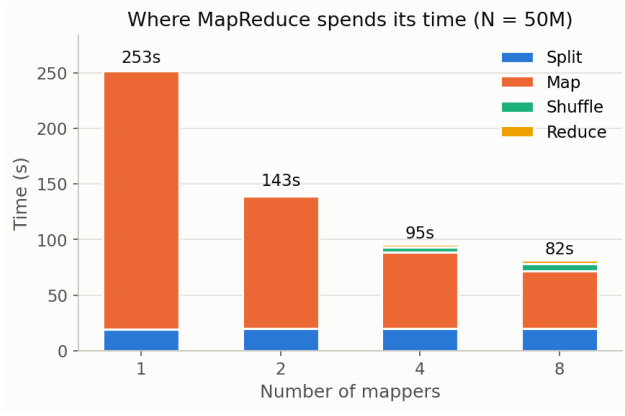
Figure 2: Speedup of each implementation compared with its own 1-worker run. Dashed line: ideal speedup.

MapReduce gets faster with more mappers, but not 8 times faster (Fig. 2). In MapReduce each mapper reads and processes its own piece, so the slow part is done in parallel. At 50M records MapReduce is 3.1 times faster with 8 mappers than with 1. It does not reach 8 times because:

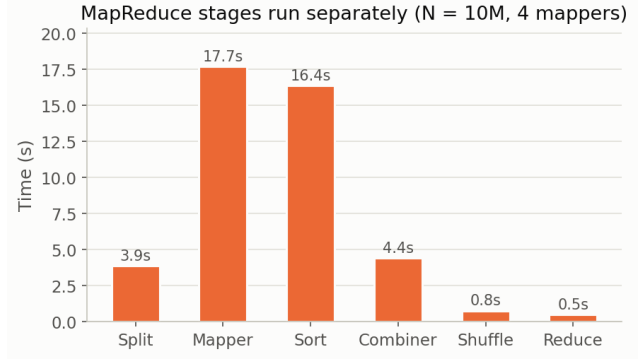
- **Splitting the input is done by one process** and always takes about 20s at 50M. That is 8% of the time with 1 mapper but 24% with 8 mappers. (In real Hadoop this step does not exist, because HDFS already stores the file in pieces.)
- **Shuffle and reduce get slower with more mappers** (2s with 1 mapper, 9.5s with 8), because more mappers send more lines to the reducer (explained below).
- **All 8 mappers share one node**, so they compete for the same memory and disk.

Where the time goes (Fig. 3a, 3b). The map phase (mapper + sort + combiner) takes almost all the time: 232 of 253s with 1 mapper at 50M. To see what is inside it, we also ran the three parts separately (10M records, 4 mappers): the mapper took 17.7s, the sort 16.4s and the combiner only 4.4s. So most of the time is spent on **converting records to text and sorting them**, not on the actual analytics. The mapper is slow because it splits every line into words and writes 1.5 GB of text for 10M records.

More mappers send more data to the reducer (Fig. 4a). With 1 mapper at 50M, 8.3M lines reach the reducer; with 8 mappers, 35.2M lines. The reason: each mapper’s combiner can only merge the lines of its *own*

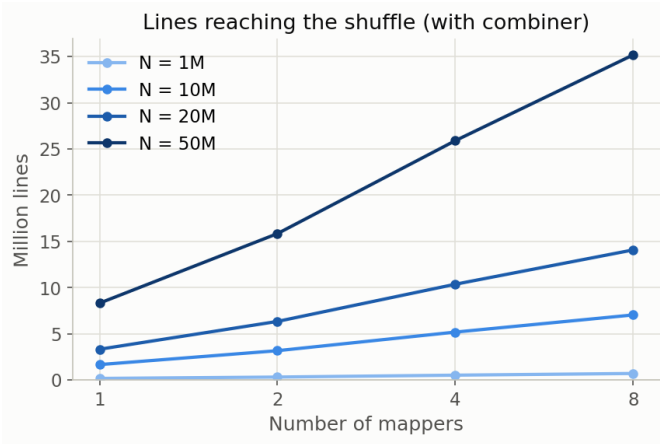


(a) Time of each stage, 50M records.

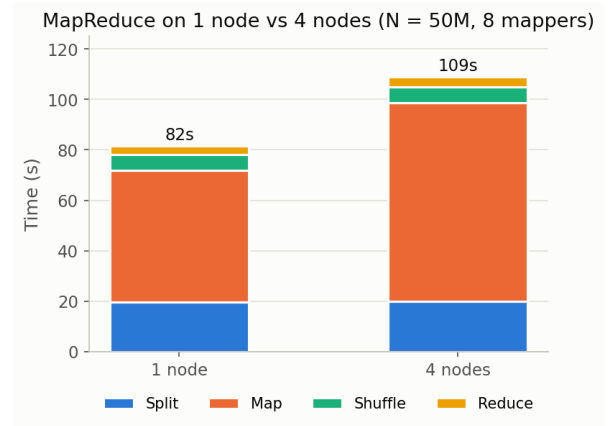


(b) Map phase split into parts, 10M records, 4 mappers.

Figure 3: Where MapReduce spends its time.



(a) Lines sent to the reducer.



(b) 1 node vs 4 nodes, 50M records, 8 mappers.

Figure 4: Data sent to the reducer, and running on more nodes.

piece. If the records of one interval are spread over 5 pieces, that interval is sent 5 times. Almost all of these lines are interval lines: there are about 8.3M intervals with only about 6 records each, so they get spread over many pieces. We checked this with a simple formula: an interval with on average 6 records, split randomly over P pieces, appears in $P(1 - e^{-6/P})$ pieces. This predicts 8.31M, 15.84M, 25.90M and 35.18M lines for 1, 2, 4 and 8 mappers, which matches our measurements.

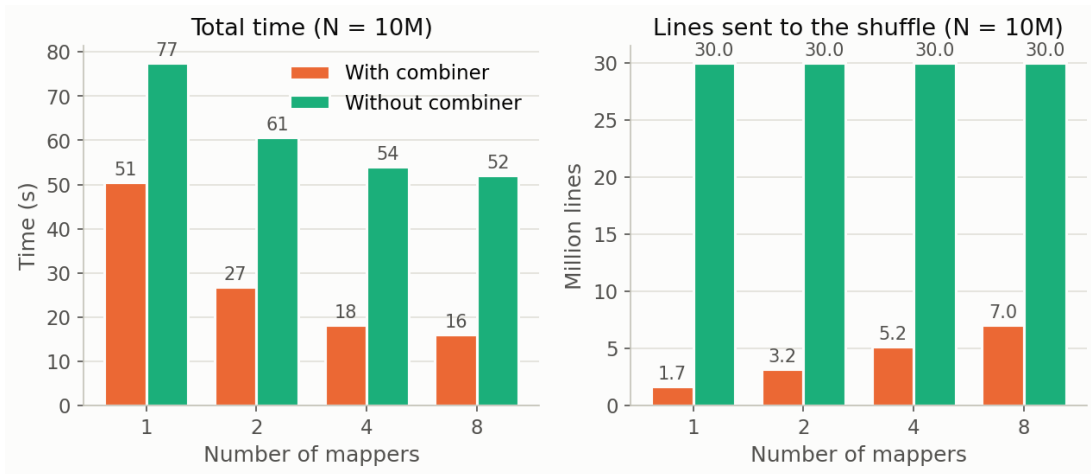


Figure 5: MapReduce with and without the combiner, 10M records.

The combiner makes a big difference (Fig. 5). Without the combiner, all 3 lines of every record go to the one reducer: 30M lines at 10M records. The shuffle then takes about 15s and the reducer 13s, instead of less than 1.3s each. With 8 mappers the whole run is 3.2 times faster with the combiner (16s vs 52s). Without it, adding mappers barely helps, because the single reducer becomes the bottleneck.

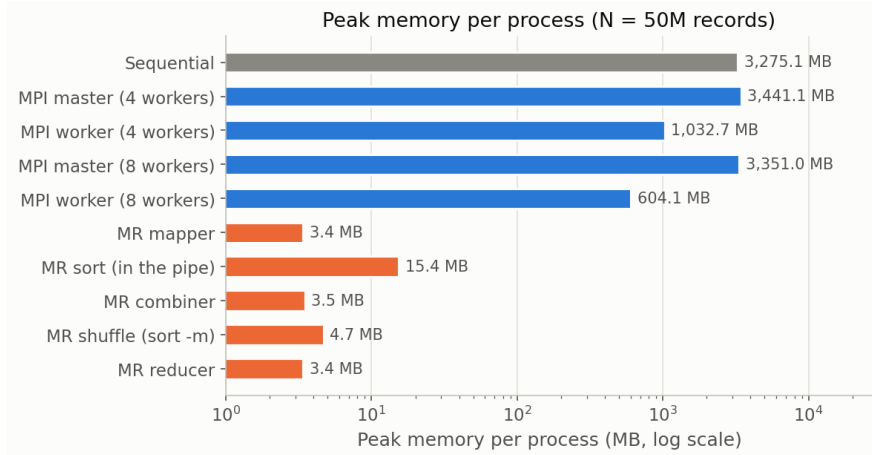


Figure 6: Peak memory of each process, 50M records (log scale).

MapReduce uses much less memory (Fig. 6). This is where MapReduce clearly wins. In MPI, rank 0 reads *all* the records into memory before sending them out: 50M records $\times$ 56 bytes is about 2.8 GB, plus the hash map of all the intervals. So rank 0 used 3.4 GB and each worker around 1 GB, which is 7.6 GB for the whole run with 4 workers. Our MapReduce programs only look at one line at a time: the mapper, combiner and reducer each used about 3.5 MB, and **sort** used about 15 MB (it writes to temporary files on disk when needed). The whole MapReduce run used only 91 MB, about **80 times less**. This means MPI cannot handle data bigger than one machine’s memory, while MapReduce can.

Running on 4 nodes was slower (Fig. 4b). We also spread the 8 mappers over 4 nodes. This made the run slower (82s $\rightarrow$ 109s); only the map phase got slower. Our explanation: the input pieces are stored in our home folder, which is a shared network drive. On one node, the pieces were just written by the same node and are still in its memory, so reading them is fast. On 4 nodes, the other 3 nodes must download their pieces over the network at the same time. Real Hadoop avoids this with HDFS: the pieces are stored on the worker nodes’ own disks, and each map task runs on the node that already has its piece (*data locality*).

1.7 MPI vs MapReduce: summary

	MPI (HW2)	MapReduce
Time at 50M (best)	55.8 s	81.5 s
Speedup, 1 $\rightarrow$ 8 workers	none (rank 0 reads the input alone)	3.1$\times$
Memory at 50M	7.6 GB (3.4 GB on rank 0)	91 MB
Data sent	binary structs	text lines, reduced by the combiner
Code we had to write	custom MPI datatype, scatter, 9 different send/receive messages, merging	3 small programs, no communication code
Adding a new statistic	change the struct, the messages and the merge	add one new key
If a process crashes	the whole job fails	Hadoop re-runs only that task

Conclusion. For our 2 GB input on one machine, MPI is faster because it works on binary data in memory. MapReduce is slower because of all the text conversion and sorting, but it gets faster with more mappers, uses 80 times less memory, and was much simpler to write. For data that does not fit on one machine, MapReduce is the better choice.

2 Q2: Real-time streaming analytics using gRPC

To be added.